Spark RDD és DataFrame feladatok

https://gemini.google.com/app/3e541b854792c161

*User prompt: # Spar context létrehozása.# Mindenképp csak egyszer futtassuk le!!!# Ezen keresztül tudjuk elérni az RDD-kehz tartozó low level API-t.from pyspark import SparkConffrom pyspark import SparkContextconf = SparkConf()sc = SparkContext(conf=conf) Spark RDD feladatok 1. feladat (2 pont) A vasarlasok.txt egy kiskereskedelmi áruházban megvásárolt termékeket tartalmazzák. Minden sor egy vásárló kosarában lévő termékeket sorolja fel, vesszővel elválasztva. Határozd meg azt a három terméket, amelyekből a legtöbbet vásárolták és add meg hányat. Figyelj oda arra, hogy az adathalmazban kis és nagy betűk is előfordulhatnak. Azaz előfordulhat a "rizs" és a "Rizs" termék is. Ezeket azonos termékeknek kell tekintened. Egy lehetséges kimenet: ('kenyer', 108), ('cukor', 68), ('tej', 64) lines = sc.textFile("vasarlasok.txt")\.flatMap(lambda l : l.split(","))\.map(lambda w: w.lower())\.map(lambda w : (w,1))\.reduceByKey(lambda a, b: a + b)\.sortBy(lambda x: x[1], False)print(lines.take(3)) [('kenyer', 108), ('cukor', 68), ('tej', 64)] 2. feladat (2 pont) Az online_retail_data.csv egy webáruház eladásait tartalmazza. Határozd meg, hogy melyik terméket melyik termékkel vásárolják gyakran együtt Franciaországban („France”). Azaz melyek a leggyakrabban előforduló termékpárok. Add meg a párok előfordulásának a számát és a 3 legtöbbször előfordulót ad vissza. Egy lehetséges kimenet: [(('POSTAGE', 'RABBIT NIGHT LIGHT'), 65), (('POSTAGE', 'RED TOADSTOOL LED NIGHT LIGHT'), 62), (('PLASTERS IN TIN CIRCUS PARADE ', 'POSTAGE'), 58)] retailFile = 'online_retail_data.csv'retailRDD = sc.textFile(retailFile)def createPairs(x): pairs = [] for i in list(x[1]): for j in list(x[1]): if i<j: pairs.append((i,j)) return pairs ret2 = retailRDD.filter(lambda line : not line.startswith("InvoiceNo")).filter(lambda line: line.split(",")[-1] == "France")res = ret2.map(lambda line: (line.split(",")[0],line.split(",")[2]))\.groupByKey().map(createPairs).flatMap(lambda x:x).map(lambda x: (x,1))\.reduceByKey(lambda a,b: a+b).sortBy(lambda x: x[1], False).take(3)print(res) [(('POSTAGE', 'RABBIT NIGHT LIGHT'), 65), (('POSTAGE', 'RED TOADSTOOL LED NIGHT LIGHT'), 62), (('PLASTERS IN TIN CIRCUS PARADE ', 'POSTAGE'), 58)] Spark DataFrame feladatok Az alábbi feladatok megoldásához a menu.csv és MenuCategory.csv fájlokat kell beolvasni. Az adathalmazok gyorséttermi menükről és azok kategóriáiról tartalmaz információkat. from pyspark.sql import SparkSessionfrom pyspark.sql import *from pyspark.sql.functions import *spark = SparkSession.builder.getOrCreate() menu = spark.read.format("csv")\.option('header', True)\.option("inferschema", True)\.load("menu.csv")menuCat = spark.read.format("csv")\.option('header', True)\.option("inferschema", True)\.load("menuCategory.csv")#menu.createOrReplaceTempView("menu_table")#menuCat.createOrReplaceTempView("menuCategory_table' 3. feladat (1 pont) Melyek azok az ételek, amelyek meghaladják az ajánlott napi zsír bevitelt? (Total Fat (% Daily Value)) Elvárt oszlopok: [Item] menu.select(col("Item"))\.where(col("Total Fat (% Daily Value)")>100)\.show(20,False) +----------------------------+ |Item | +----------------------------+ |Chicken McNuggets (40 piece)| +----------------------------+ 4. feladat (1 pont) Melyik ételnek van a maximális Sugars értéke? Elvárt oszlopok: [Item, Sugars] maxSugar = menu.select(max(col("Sugars"))).head()[0]menu.where(col("Sugars") == maxSugar)\.select("Item", "Sugars")\.show(20,False) +-----------------------------------+------+ |Item |Sugars| +-----------------------------------+------+ |McFlurry with M&M’s Candies (Medium)|128 | +-----------------------------------+------+ 5. feladat (1 pont) Hány elem van kategóriánként? Rendezzük csökkenő sorrendbe és adjuk meg a kategóriák nevét is. Elvárt oszlopok: [Name, Cnt] menu.join(menuCat, col("Category") == col("Id"))\.groupBy("Name")\.agg(count("*").alias("cnt"))\.orderBy(desc("cnt"))\.show() +------------------+---+ | Name|cnt| +------------------+---+ | Coffee & Tea| 95| | Breakfast| 42| |Smoothies & Shakes| 28| | Chicken & Fish| 27| | Beverages| 27| | Beef & Pork| 15| | Snacks & Sides| 13| | Desserts| 7| | Salads| 6| +------------------+---+ 6. feladat (1 pont) Átlagosan mennyi kalóriát tartalmaznak az egyes kategóriák? Adjuk meg a kategória nevét és rendezzük átlag alapján csökkenő sorrendbe. A 'Coffee and Tea' és a 'Beverages' kategóriákat ne vegyük figyelembe. Elvárt oszlopok: [Name, AvgCal] menu.join(menuCat, col("Category") == col("Id"))\.where((col("Name") != "Coffee and Tea") & (col("Name") != 'Beverages'))\.groupBy("Name")\.agg(round(avg(col("Calories"))).alias("avgCal"))\.orderBy(desc("avgCal"))\.show() +------------------+------+ | Name|avgCal| +------------------+------+ | Chicken & Fish| 553.0| |Smoothies & Shakes| 531.0| | Breakfast| 527.0| | Beef & Pork| 494.0| | Coffee & Tea| 284.0| | Salads| 270.0| | Snacks & Sides| 246.0| | Desserts| 222.0| +------------------+------+ magyarázdd el részletesen mi mit csinál*

Response: Az alábbiakban sorról sorra, lépésről lépésre részletesen elmagyarázom a megadott PySpark RDD és DataFrame feladatok megoldásait.

0. Előkészületek (Spark Context létrehozása)

```
from pyspark import SparkConf
from pyspark import SparkContext

conf = SparkConf()
sc = SparkContext(conf=conf)
```

- **`SparkConf()`**: Létrehozza a Spark konfigurációs objektumát, ahol beállíthatnánk az alkalmazás nevét, a használandó erőforrásokat stb.
- **`SparkContext(conf=conf)`**: A `SparkContext` (röviden `sc`) az belépési pont a low-level Spark RDD API-hoz. Létrehozza a kapcsolatot a Spark felhővel/klaszterrel. Ez felelős az RDD-k létrehozásáért és kezeléséért.

Spark RDD feladatok

1. feladat – Leggyakrabban vásárolt termékek (2 pont)

A feladat célja: Megkeresni a 3 leggyakrabban előforduló terméket a `vasarlasok.txt` fájlból, figyelmen kívül hagyva a kis- és nagybetűs eltéréseket.

```
lines = sc.textFile("vasarlasok.txt") \
    .flatMap(lambda l: l.split(",")) \
    .map(lambda w: w.lower()) \
    .map(lambda w: (w, 1)) \
    .reduceByKey(lambda a, b: a + b) \
    .sortBy(lambda x: x[1], False)

print(lines.take(3))
```

Működés lépésről lépésre:

1. **`sc.textFile("vasarlasok.txt")`**: Beolvassa a fájlt egy RDD-be, ahol minden egyes elem a fájl egy-egy sora.
2. **`.flatMap(lambda l: l.split(","))`**:
    - A `.split(",")` feldarabolja az adott sort a vesszők mentén elemek listájára.

- A `flatMap` abban különbözik a sima `map`-től, hogy a kapott listákat "kilapítja" (*flatten*), így különálló elemekként adja vissza a termékeket egyetlen közös RDD-ben.

3. **`.map(lambda w: w.lower())`**: Minden egyes terméknél eltávolítja a szóközöket/átalakítja a karaktereket kisbetűsre, hogy a pl. `"Rizs"` és `"rizs"` azonosnak számítson.
4. **`.map(lambda w: (w, 1))`**: Kulcs-érték párokat alakít ki `(termék_név, 1)` formátumban. Minden előfordulást egy `1`-es értékkel lát el.
5. **`.reduceByKey(lambda a, b: a + b)`**: Az azonos kulccsal (terméknévvel) rendelkező párokat összevonja, és az értékeket (`1`-eseket) összeadja. Így megkapjuk a termékek összesített darabszámát: `(termék_név, összes_darabszám)`.
6. **`.sortBy(lambda x: x[1], False)`**: Rendezi a párokat az érték (a darabszám, ami az `x[1]`) alapján. A `False` paraméter jelzi, hogy **csökkenő** sorrendben rendezzen.
7. **`print(lines.take(3))`**: Kigyűjti és kiírja a rendezett RDD legelső 3 elemét a driver programba.

2. feladat – Leggyakoribb termékpárok Franciaországban (2 pont)

A feladat célja: Megkeresni azokat a termékpárokat, amelyeket a franciaországi (`"France"`) vásárlások során a legtöbbször vettek együtt.

```
retailFile = 'online_retail_data.csv'
retailRDD = sc.textFile(retailFile)

def createPairs(x):
    pairs = []
    for i in list(x[1]):
        for j in list(x[1]):
            if i < j:
                pairs.append((i, j))
    return pairs

ret2 = retailRDD.filter(lambda line: not line.startswith("InvoiceNo")) \
                .filter(lambda line: line.split(",")[-1] == "France")

res = ret2.map(lambda line: (line.split(",")[0], line.split(",")[2])) \
    .groupByKey() \
    .map(createPairs) \
    .flatMap(lambda x: x) \
    .map(lambda x: (x, 1)) \
    .reduceByKey(lambda a, b: a + b) \
    .sortBy(lambda x: x[1], False) \
    .take(3)

print(res)
```

Működés lépésről lépésre:

1. **`createPairs(x)` függvény**:
   - A bemenete egy `(InvoiceNo, [termék1, termék2, ...])` pár.
   - Két egymásba ágyazott ciklussal képezi a kosárban lévő termékek párjait.
   - Az `if i < j` feltétel két dolgot biztosít:
     - Elkerüli, hogy egy terméket önmagával párosítson (`i == j`).
     - Elkerüli a duplikációkat és a sorrendi eltéréseket: pl. az `('A', 'B')` és `('B', 'A')` közül csak az egyik jön létre (mindig az ábécé szerinti kisebb lesz az első), így azonos párokként számolódnak majd.
2. **`ret2` szűrése**:
   - **`.filter(lambda line: not line.startswith("InvoiceNo"))`**: Kiszűri a CSV fájl fejlétsorát.
   - **`.filter(lambda line: line.split(",")[-1] == "France")`**: Megtartja azokat a sorokat, ahol a CSV utolsó oszlopa (ország) `"France"`.
3. **`res` feldolgozás**:
   - **`.map(lambda line: (line.split(",")[0], line.split(",")[2]))`**: Kigyűjti a tranzakció azonosítóját (0. oszlop: InvoiceNo) és a termék nevét (2. oszlop: Description), létrehozva `(InvoiceNo, Description)` párokat.
   - **`.groupByKey()`**: Csoportosítja a termékeket számlaszám (`InvoiceNo`) szerint: `(InvoiceNo, [termék1, termék2, ...])`.
   - **`.map(createPairs)`**: A kosár tartalmából előállítja a termékpárok listáját: `[(termékA, termékB), (termékA, termékC), ...]`.
   - **`.flatMap(lambda x: x)`**: A kosáronkénti termékpár-listákat kilapítja egyetlen nagy termékpár-folyammá.
   - **`.map(lambda x: (x, 1))`**: Minden termékpárt kiegészít egy 1-essel: `((termékA, termékB), 1)`.
   - **`.reduceByKey(lambda a, b: a + b)`**: Összeadja az azonos termékpárok előfordulásait.
   - **`.sortBy(lambda x: x[1], False)`**: Csökkenő sorrendbe rendezi a párokat a gyakoriság alapján.

- `.take(3)`: Visszaadja a 3 leggyakoribb termékpárt.

Spark DataFrame feladatok

DataFrame előkészítés

```
from pyspark.sql import SparkSession
from pyspark.sql import *
from pyspark.sql.functions import *

spark = SparkSession.builder.getOrCreate()

menu = spark.read.format("csv") \
    .option('header', True) \
    .option("inferschema", True) \
    .load("menu.csv")

menuCat = spark.read.format("csv") \
    .option('header', True) \
    .option("inferschema", True) \
    .load("menuCategory.csv")
```

- **`SparkSession.builder.getOrCreate()`**: Létrehozza (vagy lekéri) a DataFrame-ek használatához szükséges Spark Session-t.
- **`spark.read.format("csv")`**: Bemeneti formátumnak a CSV-t adja meg.
- **`.option('header', True)`**: Jelzi, hogy az első sor tartalmazza az oszlopneveket.
- **`.option("inferschema", True)`**: A Spark automatikusan felismeri az oszlopok adattípusait (pl. szám, szöveg) ahelyett, hogy mindent szövegként kezelne.
- **`.load(...)`**: Beolvassa a fájlokat DataFrame-ekbe (`menu` és `menuCat`).

3. feladat – Ajánlott napi zsírbefogadást meghaladó ételek (1 pont)

A feladat célja: Listázni azokat az ételeket, ahol a napi ajánlott zsírbeviteli érték százaléka meghaladja a 100%-ot.

```
menu.select(col("Item")) \
    .where(col("Total Fat (% Daily Value)") > 100) \
    .show(20, False)
```

Működés lépésről lépésre:

1. **`menu.select(col("Item"))`**: Kiválasztja az `Item` (étel neve) oszlopot.
2. **`.where(col("Total Fat (% Daily Value)") > 100)`**: Leszűri a sorokat azokra, ahol a `Total Fat (% Daily Value)` oszlop értéke szigorúan nagyobb, mint 100.
3. **`.show(20, False)`**: Megjeleníti az első 20 találatot. A `False` paraméter megakadályozza, hogy a Spark levágja (*truncate*) a hosszú szöveges értékeket az oszlopokban.

4. feladat – Maximális cukortartalmú étel (1 pont)

A feladat célja: Megtalálni a legmagasabb cukortartalmú étele(ke)t és kiírni a nevüket, valamint a cukortartalmukat.

```
maxSugar = menu.select(max(col("Sugars"))).head()[0]

menu.where(col("Sugars") == maxSugar) \
    .select("Item", "Sugars") \
    .show(20, False)
```

Működés lépésről lépésre:

1. **`menu.select(max(col("Sugars")))`**: Kiszámítja a teljes táblázatra a `Sugars` oszlop maximális értékét. Ez egy 1 soros, 1 oszlopos DataFrame-et ad vissza.
2. **`.head()[0]`**:
   - A `.head()` visszaadja az első sort (`Row` objektum).
   - A `[0]` indexeléssel kiemeljük magát a konkrét számértéket (pl. `128`) a `Row`-ból és elmentjük a `maxSugar` változóba.
3. **`menu.where(col("Sugars") == maxSugar)`**: Leszűri a `menu` DataFrame-et azokra a sorokra, ahol a cukor értéke megegyezik az imént kiszámolt maximummal.
4. **`.select("Item", "Sugars")`**: Csak az elvárt két oszlopot (`Item`, `Sugars`) tartja meg.
5. **`.show(20, False)`**: Eredmény megjelenítése levágás nélkül.

5. feladat – Elemek száma kategóriánként (1 pont)

A feladat célja: Összekapcsolni a két táblát, és megadni a kategóriák nevét, valamint az abban található elemek számát csökkenő sorrendben.

```
menu.join(menuCat, col("Category") == col("Id")) \
    .groupBy("Name") \
    .agg(count("*").alias("cnt")) \
    .orderBy(desc("cnt")) \
    .show()
```

Működés lépésről lépésre:

1. **`menu.join(menuCat, col("Category") == col("Id"))`**: Összekapcsolja (Inner Join) a `menu` táblát a `menuCat` táblával ott, ahol a `menu.Category` oszlop értéke megegyezik a `menuCat.Id` oszlop értékével.
2. **`.groupBy("Name")`**: Csoportosítja a rekordokat a kategória neve (`Name`) alapján.
3. **`.agg(count("*").alias("cnt"))`**:
   - Az `.agg()` függvénnyel aggregációt végzünk a csoportokon.
   - A `count("*")` megszámolja a csoportba tartozó elemek/sorok számát.
   - Az `.alias("cnt")` átnevezi a kapott eredményoszlopot `cnt`-re.
4. **`.orderBy(desc("cnt"))`**: Csökkenő sorrendbe rendezi az eredményt a darabszám (`cnt`) alapján.
5. **`.show()`**: Kiírja a táblázatot.

6. feladat – Átlagos kalória kategóriánként, kivételekkel (1 pont)

A feladat célja: Kiszámolni a kategóriák átlagos kalóriatartalmát, kizárva a 'Coffee and Tea' és 'Beverages' kategóriákat, az átlag szerint csökkenő sorrendben.

```
menu.join(menuCat, col("Category") == col("Id")) \
    .where((col("Name") != "Coffee and Tea") & (col("Name") != 'Beverages')) \
    .groupBy("Name") \
    .agg(round(avg(col("Calories"))).alias("avgCal")) \
    .orderBy(desc("avgCal")) \
    .show()
```

Működés lépésről lépésre:

1. **`menu.join(menuCat, col("Category") == col("Id"))`**: Összekapcsolja a két táblát a kategória-azonosítók alapján.
2. **`.where((col("Name") != "Coffee and Tea") & (col("Name") != 'Beverages'))`**:
   - Kizárja a keresésből azokat a sorokat, ahol a kategória neve `"Coffee and Tea"` vagy `"Beverages"`.
   - PySpark-ban a logikai ÉS kapcsolathoz az `&` operátort használjuk, és a feltételeket külön zárójelbe **kell** tenni.
3. **`.groupBy("Name")`**: Csoportosít a kategórianév szerint.
4. **`.agg(round(avg(col("Calories"))).alias("avgCal"))`**:
   - `avg(col("Calories"))`: Kiszámítja a kalóriák átlagát a csoporton belül.
   - `round(...)`: Egész értékre kerekíti az átlagot (vagy tizedesjegyre, ha megadnánk második paramétert).
   - `.alias("avgCal")`: Átnevezi a kimeneti oszlopot `avgCal`-ra.
5. **`.orderBy(desc("avgCal"))`**: Az átlagos kalória szerint csökkenő sorrendbe rendezi az eredményt.
6. **`.show()`**: Kiírja az eredményt.

   Megjegyzés a kimenethez: A mintakimenetben látható egy elírási hiányosság az adathalmazban/kódban: A kódban lévő feltétel `"Coffee and Tea"`-re szűr (és-sel leírva), míg az adatbázisban a kategória neve `"Coffee & Tea"` (és-jellel). Ezért jelenik meg a kimeneti táblázatban mégis a Coffee & Tea sor. A helyes szűréshez a `col("Name") != "Coffee & Tea"` feltételt kellene használni.
